

AI-First Commerce Operations Investigator

Updated Final Solution, PostgreSQL Schema Proposal, Walkthrough,
and Phase-by-Phase Implementation Plan

Decision	Confirmed direction
Approved workflow	Explain why a paid order has not reached shipment creation
Safety boundary	Read commerce state; propose a next step; create a human-review escalation; never execute an operational change
Schema gate	Share and approve the scoped PostgreSQL schema before application, AI, or MCP implementation
Primary interface	Standard remote TypeScript MCP server
System of record	PostgreSQL with separate commerce and operations schemas
Implementation style	Bun + Turborepo + TypeScript + Node.js/Express + Tailwind trace view

Prepared for the Diligence AI take-home assignment
Updated 29 July 2026

How to use this plan

- First approve the workflow contract.
- Next share the scoped PostgreSQL schema with the client and wait for approval.
- Implement one phase at a time. Do not let a coding agent continue automatically.
- At the end of each phase, run the listed checks, inspect the output, update AGENTS.md, README, and docs/evaluations/phase-XX.md, then accept or revise the phase.
- The remote MCP workflow is the product. The Tailwind trace view is a small read-only audit aid and must be built after domain and MCP correctness.

Schema-first implementation gate

Before application, AI, or MCP implementation begins, share the proposed PostgreSQL entities, relationships, statuses, invariants, permissions, indexes, and rationale. The approved schema becomes the source of truth for later Prisma models and migrations.

1. Confirmed product scope

The client confirmed the investigation, evidence handling, recommendation, human escalation, PostgreSQL persistence, standard MCP protocol, and a small read-only trace endpoint or view. The important revision is that the system must never reroute, retry, reserve, release, or otherwise change the operational commerce state.

Area	Confirmed behaviour
Investigation	Explain why a paid order has not reached shipment creation.
Evidence	Check order, payment, inventory, fulfilment, operational events, and shipment evidence. Return missing fields explicitly.
Incomplete evidence	Use NEEDS_MORE_INFO rather than guessing. A human case may request the missing information.
Recommendation	Suggest an evidence-based next step and create a human-review escalation.
Operational mutation	Forbidden. No tool may update orders, payments, inventory, fulfilments, events, or shipments.
Workflow persistence	Investigations, immutable evidence snapshots, escalations, idempotency records, and audit activity persist in PostgreSQL across restarts.
Protocol	Use standard MCP. Demonstrate through at least one compatible host; do not claim every client was tested.
Trace view	Read-only visibility into evidence, tool activity, diagnosis, and escalation. It does not replace the MCP demo.

2. Product definition and responsibility split

The product is an AI-assisted commerce operations investigator. It answers one bounded question: "Why has this paid order not reached shipment creation?"

Operations user

-> MCP-compatible AI host

-> remote TypeScript MCP server

-> deterministic evidence and diagnosis workflow

-> PostgreSQL commerce records (read only)

-> PostgreSQL operations records (scoped writes)

Layer	Responsibility
Operations user	Asks about an order and reviews the evidence-based next step.
AI host	Understands the request, selects MCP tools, and explains structured results in business language.
MCP server	Validates inputs and exposes the approved investigation, escalation, and trace capabilities.
Evidence service	Reads all relevant sources and creates one normalized order snapshot.
Diagnosis service	Checks completeness/conflicts and applies deterministic TypeScript rules.
Workflow service	Persists investigations, snapshots, escalations, idempotency, and audit events.

Layer	Responsibility
PostgreSQL	Separates read-only operational commerce state from writable operations-workflow state.

Important wording

The LLM does not convert JSON into business logic. TypeScript applies the business rules and returns structured JSON. The LLM converts that JSON into a clear explanation.

3. Schema review gate before AI/MCP implementation

The next client-facing deliverable is the scoped PostgreSQL schema. No MCP tools, agent loop, Prisma migration, or application workflow should be implemented until the schema boundary is accepted.

- Deliver docs/database/schema-proposal.md with entities, columns, relationships, statuses, invariants, permissions, indexes, rationale, and two walkthroughs.
- Deliver docs/database/client-review-summary.md as the concise client-facing version.
- Mark Phase 1 as Awaiting client schema approval in AGENTS.md and README until the client accepts it.
- Any later deviation from the accepted schema must be documented and reviewed before a migration is changed.

4. Proposed PostgreSQL schema

Use two logical PostgreSQL schemas to make the safety boundary visible in the data model and database permissions.

Logical schema	Purpose	Runtime permission
commerce	Synthetic order, payment, inventory, fulfilment, event, and shipment state that represents operational systems.	SELECT only
operations	Investigation, evidence, escalation, idempotency, and append-only trace records created by this workflow.	Minimum required SELECT/INSERT/UPDATE; no commerce writes

4.1 Commerce entities

Entity	Key columns	Rationale
commerce.orders	id PK; status; created_at; updated_at	One synthetic order. Runtime read-only.
commerce.order_items	id PK; order_id FK; sku; quantity > 0	Items and quantities needed for fulfilment checks.
commerce.payments	id PK; order_id UNIQUE FK; status; amount; currency; observed_at	Current scoped payment state. Payment-attempt history is deferred.
commerce.warehouses	id PK; name; active	Valid inventory and fulfilment locations.
commerce.inventory_levels	warehouse_id + sku PK; available_quantity >= 0; observed_at	Current stock evidence by warehouse and SKU.
commerce.fulfilments	id PK; order_id UNIQUE FK; status; hold_reason; assigned_warehouse_id FK; timestamps	Current fulfilment state and assigned warehouse.
commerce.fulfilment_events	id PK; order_id FK; fulfilment_id nullable FK; type; status; details JSONB; occurred_at	Failure and progression evidence such as creation or label failures.
commerce.shipments	id PK; order_id UNIQUE FK; fulfilment_id nullable FK; status; created_at	Shipment existence and current scoped state.

Scoped choice: one current payment, fulfilment, and shipment row per order. This is sufficient for the assignment. Detailed provider attempt history and multiple fulfilments/shipments are future extensions; operational events preserve the failures needed by the selected scenarios.

4.2 Operations-workflow entities

Entity	Key columns	Rationale
operations.investigations	id PK; trace_id UNIQUE; order_id FK; client_request_id; status; evidence_status; diagnosis_code; confidence; matched_rule; timestamps; error_code	Owns one investigation lifecycle and searchable outcome.
operations.investigation_evidence	investigation_id PK/FK; snapshot JSONB; missing_fields; conflicts JSONB; source_observed_at JSONB; created_at	Immutable evidence exactly as observed at decision time.

Entity	Key columns	Rationale
operations.human_review_escalations	id PK; investigation_id UNIQUE FK; order_id FK; status; queue; reason_code; suggested_next_step; dedupe_key UNIQUE; timestamps	Persistent human-review handoff, derived from stored investigation.
operations.idempotency_records	tool_name + idempotency_key UNIQUE; request_hash; resource_type; resource_id; response_snapshot; created_at	Returns the same logical result after safe client retries.
operations.audit_events	id PK; event_key UNIQUE; trace_id; investigation_id; escalation_id; event_type; tool_name; status; safe input/output JSONB; error_code; duration_ms; created_at	Append-only trace of observable tool and workflow activity.

4.3 Relationships

commerce.orders	1	-----	*	commerce.order_items
commerce.orders	1	-----	1	commerce.payments
commerce.orders	1	-----	0..1	commerce.fulfilments
commerce.orders	1	-----	*	commerce.fulfilment_events
commerce.orders	1	-----	0..1	commerce.shipments
commerce.warehouses	1	--	*	commerce.inventory_levels
commerce.warehouses	1	--	*	commerce.fulfilments
commerce.orders	1	-----	*	operations.investigations
operations.investigations	1	--	1	operations.investigation_evidence
operations.investigations	1	--	0..1	operations.human_review_escalations
operations.investigations	1	--	*	operations.audit_events
operations.human_review_escalations	1	--	*	operations.audit_events

4.4 Statuses and outcome codes

Category	Values	Why separate
Investigation lifecycle	RUNNING, COMPLETED, NEEDS_MORE_INFO, FAILED	Describes workflow progress, not the business cause.
Evidence state	COMPLETE, MISSING, CONFLICTING	Controls whether a confident diagnosis is allowed.
Escalation lifecycle	AWAITING_REVIEW, IN_REVIEW, CLOSED	Tracks the human-review handoff only.
Diagnosis codes	ASSIGNED_WAREHOUSE_OUT_OF_STOCK; FULFILMENT_CREATION_FAILED; WITHIN_EXPECTED_PROCESSING_TIME; SHIPMENT_LABEL_CREATION_FAILED; SHIPMENT_ALREADY_EXISTS; PAYMENT_NOT_CONFIRMED; CAUSE_NOT_DETERMINED	Business result remains independent from investigation status.
Audit status	STARTED, SUCCEEDED, FAILED	Records each observable tool/workflow step.

4.5 Core invariants

- The runtime application role has SELECT only on commerce.*.
- Order-item quantity is positive; inventory quantity is non-negative; all source references are valid.
- An investigation must reference an existing order.
- COMPLETED requires COMPLETE evidence and a diagnosis code.
- NEEDS_MORE_INFO requires explicit missing fields or conflicts. It cannot include a specific recommendation that depends on the missing/conflicting evidence.
- Evidence snapshots are immutable after completion or NEEDS_MORE_INFO.
- A human-review escalation is derived from the stored investigation. The LLM cannot submit or overwrite diagnosis or evidence.
- Audit events are append-only and store observable activity, not hidden chain of thought.
- Idempotency is unique per tool and key. Reusing a key with a different request hash returns a conflict.
- Only one open escalation is allowed for the same investigation/dedupe key.
- No workflow transaction updates commerce orders, payments, inventory, fulfilments, events, or shipments.

4.6 Required indexes and permission checks

Query or constraint	Index / enforcement
Order investigation	commerce.orders(id); foreign keys on order_id
Trace lookup	operations.investigations(trace_id UNIQUE); audit_events(trace_id, created_at)
Investigation lookup	operations.investigations(id); investigations(order_id, created_at)
Open escalation	human_review_escalations(investigation_id UNIQUE); dedupe_key UNIQUE; status index
Idempotency	idempotency_records(tool_name, idempotency_key UNIQUE)
Audit ordering	audit_events(investigation_id, created_at, id)
Safety boundary	Database grants: commerce SELECT only; operations limited DML; integration test attempts forbidden writes and expects failure

Snapshot trade-off

Store the normalized evidence as immutable JSONB so the exact decision-time evidence survives later source changes. Keep searchable top-level investigation fields for common queries. This avoids over-normalizing an evolving evidence shape while preserving traceability.

5. Turborepo package design

apps/	
api/	Node.js + Express; /mcp, /health, trace routes
web/	minimal Tailwind trace viewer, built late
packages/	
config/	shared TypeScript, env, test configuration
schemas/	Zod schemas and public protocol/domain types
db/	Prisma, migrations, database clients, repositories
fixtures/	synthetic cases, validation, seed/reset
evidence/	collectors, source adapters, normalization
diagnosis/	readiness/conflict gate and deterministic rules
workflow/	orchestration, persistence, idempotency, escalation
mcp/	tool registration and MCP adapters
agent/	host-neutral instructions and LLM evaluation helpers
evaluations/	scenario, guardrail, contract, and model evaluations
observability/	trace event types and trace queries
docs/	
database/schema-proposal.md	
database/client-review-summary.md	
evaluations/phase-XX.md	
workflow-contract.md	
AGENTS.md	
README.md	
turbo.json	

Repository convention: Bun package manager and workspaces, strict TypeScript + ESM, no src/ directories, and small public package APIs. Core packages never import from apps, and diagnosis never imports Prisma.

6. Synthetic data and seed strategy

Order	Scenario	Evidence	Expected result	Allowed outcome
ORD-1042	Assigned warehouse out of stock	COMPLETE	ASSIGNED_WAREHOUSE_OUT_OF_STOCK	Review possible reassignment; no state change
ORD-1043	Fulfilment creation failed	COMPLETE	FULFILMENT_CREATION_FAILED	Escalate to fulfilment operations
ORD-1044	Within expected processing window	COMPLETE	WITHIN_EXPECTED_PROCESSING_TIME	Explain normal wait; no escalation by default
ORD-1045	Shipment label creation failed	COMPLETE	SHIPMENT_LABEL_CREATION_FAILED	Escalate to shipping/fulfilment operations
ORD-1046	Missing inventory evidence	MISSING	None	NEEDS_MORE_INFO; list missing fields
ORD-1047	Shipment already exists	COMPLETE	SHIPMENT_ALREADY_EXISTS	Explain stale operator view
ORD-1048	Cause not determined	COMPLETE	CAUSE_NOT_DETERMINED	Escalate without inventing a cause

Order	Scenario	Evidence	Expected result	Allowed outcome
ORD-1049	Operator says paid; source says processing	COMPLETE	PAYMENT_NOT_CONFIRMED	Trust source of truth; payment review
ORD-1050	Conflicting inventory evidence	CONFLICTING	None	NEEDS_MORE_INFO; no warehouse suggestion

6.1 Validation and seed sequence

```

Create typed fixture objects
-> validate each record with Zod
-> validate relationships across records
-> validate scenario expectations/invariants
-> open one PostgreSQL transaction
-> seed commerce tables
-> read back and verify counts/relationships
-> commit

```

- Do not use an LLM to extract or validate synthetic JSON that the project authors directly.
- Application startup must never reset or reseed PostgreSQL.
- Use explicit Bun commands for migrate, seed, and reset-demo.
- Seed only commerce records. Investigations, escalations, idempotency, and audits are created by the running workflow.

7. Detailed walkthrough: "Why is ORD-1042 not shipping?"

7.1 Starting evidence

Source	Observed evidence
Order	ORD-1042 is CONFIRMED and requires 1 unit of SKU-RED-SHOE-42.
Payment	SUCCEEDED. This is verified from the payment record, not trusted from the operator wording.
Fulfilment	ON_HOLD; reason INVENTORY_OUT_OF_STOCK; assigned to Warehouse A.
Inventory	Warehouse A has 0 units; Warehouse B has 3 units; observations include timestamps.
Shipment	No shipment row exists.
Events	FULFILMENT_CREATED followed by INVENTORY_HOLD_ADDED.

7.2 End-to-end sequence before escalation

Step	What happens
1. Operator question	The user asks the connected AI host: "Why is ORD-1042 not shipping?"
2. Tool selection	The LLM selects investigate_order_exception based on its description. It does not answer from memory.
3. Input and idempotency	The MCP validates orderId and clientRequestId. A repeated clientRequestId returns the existing investigation.
4. Investigation start	The workflow creates RUNNING investigation INV-2001, trace TRACE-2001, and an INVESTIGATION_STARTED audit event.
5. Evidence collection	Repository interfaces read order, payment, fulfilment, inventory, shipment, and operational events. Each source result and observedAt timestamp is recorded.
6. Normalization	Provider/table-specific records become one OrderInvestigationSnapshot. Warehouse eligibility is calculated deterministically for every line item.
7. Readiness gate	The system checks required fields and conflicts. ORD-1042 is COMPLETE. A missing/conflicting case would stop here and become NEEDS_MORE_INFO.
8. Diagnosis	Ordered TypeScript rules match ASSIGNED_WAREHOUSE_OUT_OF_STOCK because payment succeeded, no shipment exists, fulfilment is on an inventory hold, and the assigned warehouse has no stock.
9. Suggested action	The service may state that Warehouse B is an eligible option for human review. It does not reserve stock, reassign fulfilment, release a hold, or retry shipment creation.
10. Persistence	The investigation, immutable evidence snapshot, matched rule, diagnosis, source timestamps, and audit events are saved transactionally before the MCP returns.
11. Structured MCP output	The tool returns diagnosis, supporting evidence, missing/conflict lists, suggested human queue, traceId, investigationId, and commerceStateChanged=false.
12. LLM explanation	The host explains the facts in business language and offers to create a human-review escalation.

7.3 Structured result and business explanation

```
{
  "investigationId": "INV-2001",
  "traceId": "TRACE-2001",
  "orderId": "ORD-1042",
  "status": "COMPLETED",
  "evidenceStatus": "COMPLETE",
  "diagnosis": {
    "code": "ASSIGNED_WAREHOUSE_OUT_OF_STOCK",
    "confidence": "CONFIRMED"
  },
  "missingFields": [],
  "conflicts": [],
  "suggestedNextStep": "Review possible reassignment to Warehouse B.",
  "commerceStateChanged": false
}
```

Expected AI explanation

Payment succeeded, but fulfilment is on hold because the assigned Warehouse A has no available inventory. Warehouse B currently has enough stock and no shipment exists. No operational state was changed. A human fulfilment specialist should review whether reassignment is appropriate.

7.4 Human-review escalation

If the operator asks to escalate, the AI calls `create_human_review_escalation` with only `investigationId` and an idempotency key. The server loads the stored investigation and derives the case data. The model cannot submit a free-form diagnosis or evidence.

```
operations.human_review_escalations
case_id: CASE-2001
investigation_id: INV-2001
order_id: ORD-1042
status: AWAITING_REVIEW
queue: FULFILMENT_OPERATIONS
reason_code: ASSIGNED_WAREHOUSE_OUT_OF_STOCK
suggested_next_step: Review possible reassignment to Warehouse B
```

8. Missing or conflicting evidence flow

The evidence gate is a safety feature, not an error to hide. If required information cannot be read or conflicts, the workflow must persist the uncertainty and ask a human for more information.

```
ORD-1046
payment.status = SUCCEEDED
fulfilment.status = ON_HOLD
assigned warehouse = Warehouse A
assigned-warehouse inventory = missing
shipment exists = false

Result:
investigation.status = NEEDS_MORE_INFO
evidence.status = MISSING
missingFields = ["inventory.assignedWarehouse"]
diagnosisCode = null
suggestedNextStep = "Verify current inventory evidence before deciding the operational action."
```

- Do not claim the warehouse is out of stock when inventory cannot be read.
- Do not recommend Warehouse B when inventory observations conflict.
- A `NEEDS_MORE_INFO` escalation can be routed to an `operations/data-review` queue, but it cannot contain a guessed fix.

9. MCP capability design

Tool	Purpose	Write boundary
<code>list_demo_cases</code>	Discover synthetic orders and expected scenario category.	No writes
<code>investigate_order_exception</code>	Gather evidence, validate readiness, diagnose, persist investigation/snapshot/audit, and return a structured result.	Writes operations workflow records only; commerce is read-only
<code>create_human_review_escalation</code>	Create or return one persistent review case derived from a stored investigation.	Writes escalation/idempotency/audit only
<code>get_review_case</code>	Return the persisted escalation and source investigation.	No writes
<code>get_investigation_trace</code>	Return ordered observable tool and workflow events.	No writes

Tool descriptions must be domain-specific. Do not expose `run_sql`, `update_record`, generic API, reserve inventory, reassign fulfilment, release hold, retry shipment, or create shipment capabilities.

Protocol annotation nuance

The workflow is operationally read-only, but `investigate_order_exception` and `create_human_review_escalation` do persist operations records. Their MCP annotations must describe their real side effects accurately while clearly stating that `commerceStateChanged` is always false.

10. LLM and agent responsibilities

LLM / host may	LLM / host must not
Select an investigation or trace tool from the user request.	Invent payment, inventory, fulfilment, event, or shipment facts.
Explain structured evidence in business language.	Apply or override deterministic business rules.
Offer to create a human-review escalation.	Claim that an order was fixed, rerouted, retried, or shipped.
Refuse unsupported operational requests and explain the boundary.	Supply free-form diagnosis/evidence to the escalation tool.

The core MCP server does not require an internal Gemini call. Claude, ChatGPT, Codex, or another MCP-compatible host already provides the reasoning/explanation layer. A provider SDK may be used only in a separate evaluation harness if repeatable model testing is helpful.

11. Backend correctness and future scaling

Concern	Initial implementation	Why it scales later
Idempotency	Unique <code>tool_name</code> + <code>idempotency_key</code> , request hash, persisted response/resource.	Safe retry after client/network uncertainty and multiple app instances.
Transactions	Persist investigation + snapshot + completion audit together; escalation + audit together.	No partial workflow history.
Retries	Retry bounded transient PostgreSQL/network failures only. Do not retry invalid input, missing evidence, conflicts, or unsupported operations.	Clear failure classification avoids duplicate work.
Concurrency	Database unique constraints prevent duplicate investigations/cases. Server instances keep no important in-memory state.	Works with horizontal application scaling.
Evidence consistency	Single DB transaction for synthetic sources; record <code>observedAt</code> timestamps for every source.	Repository adapters can later call external systems and report freshness.
Repository boundary	Interfaces isolate evidence collection from Prisma and vendor schemas.	Order/payment/inventory adapters can later be replaced independently.
Audit growth	Append-only events with trace/investigation/order indexes; demo response limits.	Pagination, retention, and archival can be added later.
Model variability	Strict tool descriptions and structured outputs; outcome-based evaluations.	Different clients can vary in wording without changing correctness.

Do not add Redis, queues, Kafka, RAG, multi-agent orchestration, production authentication, or event sourcing for this assignment. Document them as future options only when the scoped workflow creates a real need.

12. Traceability and observability

Persist the visible system activity needed to reconstruct one workflow. Do not store hidden LLM chain of thought.

```
TRACE-2001
INVESTIGATION_STARTED
TOOL_CALL_STARTED: investigate_order_exception
ORDER_FETCHED
PAYMENT_FETCHED
FULFILMENT_FETCHED
INVENTORY_FETCHED
SHIPMENT_CHECKED
EVENTS_FETCHED
EVIDENCE_VALIDATED
DIAGNOSIS_MATCHED
INVESTIGATION_PERSISTED
TOOL_CALL_SUCCEEDED
HUMAN_REVIEW_CASE_CREATED
```

- Store safe input/output summaries, source timestamps, matched rule, missing fields, conflicts, duration, and error codes.
- Expose `GET /traces/:traceId`, `GET /investigations/:id`, and `GET /review-cases/:id`.

- Build the JSON endpoints first. Add a minimal Tailwind read-only view only after domain and MCP tests pass.
- The trace view supports auditability but does not replace the MCP-compatible client demonstration.

13. Evaluation strategy

Layer	Focused checks
Fixture/schema	Valid records pass; malformed records fail; relationships and expected scenario declarations are coherent.
Repositories	Correct order isolation; missing records remain missing; read-only commerce permissions are enforced.
Evidence	All sources are collected; normalized snapshot is correct; warehouse eligibility is calculated per line item.
Readiness gate	Missing/conflicting evidence returns NEEDS_MORE_INFO and blocks unsupported conclusions.
Diagnosis	Every synthetic scenario maps to the expected deterministic outcome.
Persistence	Investigation, evidence, case, idempotency, and audit survive restarts; transactions prevent partial writes.
Guardrails	No commerce mutation tool/repository/permission exists; full workflow leaves commerce rows unchanged.
MCP contract	Standard client can list tools, validate schemas, call tools, and receive structured results.
LLM behaviour	Correct tool use, grounded explanation, refusal of dangerous requests, no false claim of operational execution.
Trace	Ordered tool/workflow activity can be fetched and contains no secrets or hidden reasoning.

Each phase creates docs/evaluations/phase-XX.md with commands, expected and actual results, manual checks, guardrails, sample output, limitations, and an Accepted / Needs revision decision. AGENTS.md and the README phase table are updated before the phase stops.

14. Phase-by-phase implementation plan

Phase completion contract

Implement only one phase, run its tests, inspect its output, update AGENTS.md, README, and the phase evaluation report, then stop for review. The separate prompt file contains the full copy-paste coding instruction for each phase.

Phase 0 - Freeze workflow contract and repository rules

Document the user, problem, evidence requirements, allowed writes, forbidden operational actions, package graph, and phase review process.

Checkpoint	Required content
Output	AGENTS.md; docs/workflow-contract.md; package graph; README phase table; evaluation template.
Tests and verification	Manual safety and dependency review; every planned capability respects commerce read-only boundary.
Exit criteria	Workflow contract accepted; schema proposal can begin.

Phase 1 - Propose scoped PostgreSQL schema for client review

Design the data model before code and make the commerce/operations boundary explicit.

Checkpoint	Required content
Output	docs/database/schema-proposal.md; Mermaid/text ERD; client-review summary; statuses; invariants; indexes; permissions; rationale; phase-01 report.
Tests and verification	Entity/status consistency; ORD-1042 and NEEDS_MORE_INFO representation; no mutation path; idempotency and traceability review.
Exit criteria	Client accepts schema. No scaffold, AI, or MCP implementation before acceptance.

Phase 2 - Scaffold Bun + Turborepo workspace

Create the approved monorepo foundation without changing the accepted schema design.

Checkpoint	Required content
Output	Bun workspaces; turbo tasks; apps/api health endpoint; minimal apps/web shell; package skeletons; strict config.
Tests and verification	bun install; build; typecheck; test; GET /health; environment validation.
Exit criteria	Repository foundation and no-src convention accepted.

Phase 3 - Define Zod contracts and synthetic scenarios

Translate accepted statuses/entities into shared schemas and create ORD-1042 to ORD-1050 fixtures.

Checkpoint	Required content
Output	packages/schemas and packages/fixtures in-memory catalogue with expected evidence/diagnosis/escalation behaviour.
Tests and verification	All fixtures valid; malformed fixtures fail; unique IDs; logical timestamps; expected results declared.
Exit criteria	Scenario corpus accepted before database implementation.

Phase 4 - Implement approved PostgreSQL schema and seed

Create Prisma schema/migrations exactly from the approved proposal and seed validated commerce fixtures.

Checkpoint	Required content
Output	commerce and operations schemas; roles/grants; migration; seed/reset commands; post-seed verification.
Tests and verification	Migration from empty DB; seed transaction; relationship/count checks; restart does not reset; forbidden commerce write fails.
Exit criteria	Database model and initial data accepted.

Phase 5 - Repositories and read-only commerce boundary

Create small repository interfaces and PostgreSQL implementations for every evidence source.

Checkpoint	Required content
Output	Order, payment, inventory, fulfilment, event, shipment, and workflow repositories.
Tests and verification	Correct/missing records; order isolation; event order; runtime role cannot update commerce.
Exit criteria	Every source can be queried independently.

Phase 6 - Evidence collection and normalization

Gather all source records into one stable OrderInvestigationSnapshot.

Checkpoint	Required content
Output	Evidence collector, normalized types, source timestamps, deterministic warehouse eligibility.
Tests and verification	Snapshot for every scenario; missing source remains missing; timeout/error representation; item-level stock checks.
Exit criteria	Snapshots are understandable and provider-independent.

Phase 7 - Evidence readiness and conflict gate

Decide whether evidence is COMPLETE, MISSING, or CONFLICTING before diagnosis.

Checkpoint	Required content
Output	Required-field matrix; missing/conflict output; NEEDS MORE INFO mapping.
Tests and verification	Missing payment/inventory/shipment; conflicting inventory; no recommendation depending on absent evidence.
Exit criteria	System reliably knows when not to conclude.

Phase 8 - Deterministic diagnosis and suggested action

Apply ordered TypeScript rules only after the evidence gate.

Checkpoint	Required content
Output	Diagnosis engine, rule IDs, confidence, review queue, safe suggested next step.
Tests and verification	Table-driven tests for ORD-1042 to ORD-1050; field changes alter diagnosis correctly; no LLM involved.
Exit criteria	Correctness accepted before workflow persistence/MCP.

Phase 9 - Persistent investigation and escalation workflow

Orchestrate collection, readiness, diagnosis, persistence, idempotency, audit, and review-case creation.

Checkpoint	Required content
Output	Investigation lifecycle; immutable snapshot; audit events; escalation; idempotency records; transactions.
Tests and verification	Restart persistence; duplicate request reuse; request-hash conflict; transaction rollback; NEEDS_MORE_INFO persisted without guess.
Exit criteria	One logical workflow effect per retry.

Phase 10 - Standard remote MCP server

Expose only the approved domain capabilities through standard MCP.

Checkpoint	Required content
Output	Remote /mcp; tool schemas/descriptions; structured outputs; errors; tool contract tests.
Tests and verification	tools/list; valid/invalid calls; no commerce mutation tools; workflow-writing tools report commerceStateChanged=false.
Exit criteria	MCP workflow works from a standard test client.

Phase 11 - Agent behaviour and LLM evaluations

Define host-neutral instructions and verify tool selection/explanation/refusal behaviour.

Checkpoint	Required content
Output	Agent guidance; evaluation cases; actual model/client results; grounded response checks.
Tests and verification	ORD-1042 question; dangerous mutation request; missing evidence; duplicate escalation; no false execution claim.
Exit criteria	One real MCP host behaves safely and usefully.

Phase 12 - Trace APIs and minimal Tailwind viewer

Make one complete workflow easy to inspect without turning the project into a frontend assignment.

Checkpoint	Required content
Output	Read-only trace/investigation/case endpoints; small Tailwind view; pagination/demo limits.
Tests and verification	Ordered events; restart persistence; safe 404; missing/conflicting states visible; no hidden reasoning.
Exit criteria	Trace view supports, but does not replace, MCP demo.

Phase 13 - Hardening, deployment, and submission evidence

Deploy, rerun the full matrix, document trade-offs, and prepare README, AI worklog, and demo.

Checkpoint	Required content
Output	Hosted MCP URL; database; source repo; complete reports; final README; 4-5 minute demo script.

Checkpoint	Required content
Tests and verification	Remote client connection; restart/idempotency/immutability checks; clean clone; all documented commands and claims verified.
Exit criteria	Submission demonstrates product judgement, MCP quality, safety, correctness, and traceability.

15. Final demonstration walkthrough

1. Show the hosted /mcp endpoint and connect with the selected standard MCP-compatible host or Inspector.
2. List the tools and explain that no tool can mutate commerce state.
3. Ask: "Why is ORD-1042 not shipping?"
4. Show the investigation tool call, normalized evidence, completeness result, and deterministic diagnosis.
5. Explain the cause versus the human-review suggestion.
6. Create the human-review escalation and repeat the request to demonstrate idempotency.
7. Open TRACE-2001 through the endpoint/view and show ordered evidence and tool activity.
8. Ask: "Move it to Warehouse B and retry shipment." Show that no such capability exists and the host explains the safety boundary.
9. Run ORD-1046 or ORD-1050 to demonstrate NEEDS_MORE_INFO rather than guessing.
10. Show that investigations, escalations, and audits still exist after restart and that commerce rows did not change.

16. Final completion checklist

Area	Completion condition
Schema	Client-approved entities, relationships, statuses, invariants, permissions, indexes, and rationale are the implementation source of truth.
Evidence	Order, payment, inventory, fulfilment, event, and shipment sources are collected and normalized.
Safety	Missing/conflicting evidence returns NEEDS_MORE_INFO; no commerce mutation path exists.
Diagnosis	Every synthetic scenario has a deterministic expected result and passing evaluation.
Persistence	Investigations, evidence, escalations, idempotency, and audits survive restarts.
MCP	Standard remote tools have precise schemas, descriptions, structured output, and no unsafe capabilities.
LLM	The host explains evidence and refuses unsupported operational actions; it does not invent business logic.
Trace	One workflow can be reconstructed from observable events without storing hidden reasoning.
Documentation	AGENTS.md, README, phase reports, AI worklog, architecture notes, and limitations match actual implementation.
Demo	The hosted workflow is understandable and testable without local setup.

Build priority

Schema approval -> deterministic evidence and diagnosis -> safety and persistence -> MCP quality -> LLM behaviour -> trace view -> deployment polish. Do not reverse this order.